UNITED INTERNATIONAL UNIVERSITY (UIU)
Dept. of Computer Science & Engineering
Trimester: Summer 2026 (Mid)
Course No: CSE 3812 **Title:** Artificial Intelligence Laboratory
Section: A **Time:** 60 mins **Marks:** 30

Instructions:

- While answering the questions make sure to follow this convention,
  - First create a new folder in any directory on your current PC where you want to write your code. The folder name will be your rollNumber_Mid e.g. for me, it will be 1907016_Mid
  - Then go into the folder. The folder will contain both the python scripts as answers to the two questions provided.
  - Suppose for me, my roll was 1907016. So, the first filename should be `1907016_a.ipynb` and the second one should be `1907016_b.ipynb`. For you, only the roll number part will change. `[If someone wants to write in a (dot)py script, that's completely fine too]`
  - Properly save the scripts at regular intervals if you're coding on an offline basis because unlike google colab, offline codes sometimes don't have autosave option on.
  - If you want to code online in Google colab, then you should create a new notebook after going into https://colab.research.google.com/ following the same name convention.
  - After you're done with the coding part, you should download the notebook from the `File` option on the top left corner as an ipynb file in the folder directory that I instructed as before.
- If you followed the instructions properly, now your folder directory has two ipynb/py files in it.
- Then, upload the whole folder in this Link.

GitHub Repo Link: https://github.com/IceColdMartini/262-AI-Lab-A

Go to the Repo, read the `readme.md` and you will find every answer to all your questions.

Aliceland Relief Rover

Allocated Marks: 15

In the highland regions of ***Aliceland***, emergency medical supplies are delivered using autonomous quadcopter drones. The mountainous region is modeled as a 2D integer matrix grid of size **`RxC`**, where each cell coordinate `(r,c)` represents a regional flight sector:

- `-1`: Impassable Mountain Peak / Storm Cloud (Flight prohibited).
- `≥0` : Flight sector with an integer elevation value indicating altitude in hundreds of meters (e.g., 0, 1, 2, 4).

The drone moves in **4 cardinal directions (Up, Down, Left, Right)**. The energy expenditure required to fly from a current sector `(r,c)` to an adjacent valid sector `(nr,nc)` depends dynamically on the elevation difference $\Delta E = \text{grid}[nr][nc] - \text{grid}[r][c]$:

1. **Level or Downhill Flight ($\Delta E \leq 0$):** The drone coasts or descends smoothly. The movement step cost is fixed at `1.0`.
2. **Uphill Climbing Flight ($\Delta E > 0$):** Climbing against gravity requires substantial engine thrust. The movement step cost is calculated as:

$$\text{step\_cost} = 1.0 + 2.0 \times (\text{grid}[nr][nc] - \text{grid}[r][c])$$

Implement **`solve_aliceland_drone(grid, start, goal)`** by completing the missing logic in the template. The function must return the **minimum-energy flight path as a list of coordinates and the total cumulative energy cost**.

A helper code has been provided in the github directory where almost everything is filled up except for the part which you are supposed to modify. If you test your code by executing Cell-5 of the ipynb on the sample data, it will give you an output like this:

```
Discovered Drone Flight Path: [(0, 0), (0, 1), (1, 1), (1, 2), (1, 3), (2, 3), (3, 3)]
Total Energy Cost: 10.0
```

Karp City Hovercraft

Allocated Marks: 15

In *Karp City*, municipal transportation engineers have upgraded emergency delivery units to Mark-II Omnidirectional Hovercrafts. Unlike older models restricted to 4 cardinal directions, Mark-II vehicles can travel in **8 directions** (4 Cardinal + 4 Diagonal) across the city grid matrix:

- `0`: Open Street.
- `1`: Impassable Building Structure.

Movement mechanics and energy costs are governed as follows:

1. **Cardinal Moves (Up, Down, Left, Right):** Orthogonal transitions between adjacent cells incur a standard unit cost of `1.0`.
2. **Diagonal Moves (Up-Left, Up-Right, Down-Left, Down-Right):** Diagonal transitions allow direct shortcuts across intersections but consume additional maneuvering power, incurring a step cost of `1.5`.

3. An admissible and consistent heuristic for `8-directional motion` with diagonal step cost `1.5` and cardinal cost `1.0` is the **Octile Distance**, computed as:

$$h(n) = 1.5 \times \min(|r_n - r_g|, |c_n - c_g|) + 1.0 \times \big||r_n - r_g| - |c_n - c_g|\big|$$

Implement **`solve_karp_octile_hovercraft(grid, start, goal)`** by completing the missing neighbor generation and cost integration.

A helper code has been provided in the github directory where almost everything is filled up except for the part which you are supposed to modify. If you test your code by executing Cell-5 of the ipynb on the sample data, it will give you an output like this:

```
Hovercraft Mark-II Path: [(0, 0), (1, 0), (2, 1), (3, 2), (4, 3), (4, 4)]
Total Traversal Cost: 6.5
```

HAPPY CODING

<____| 🌷 |———>